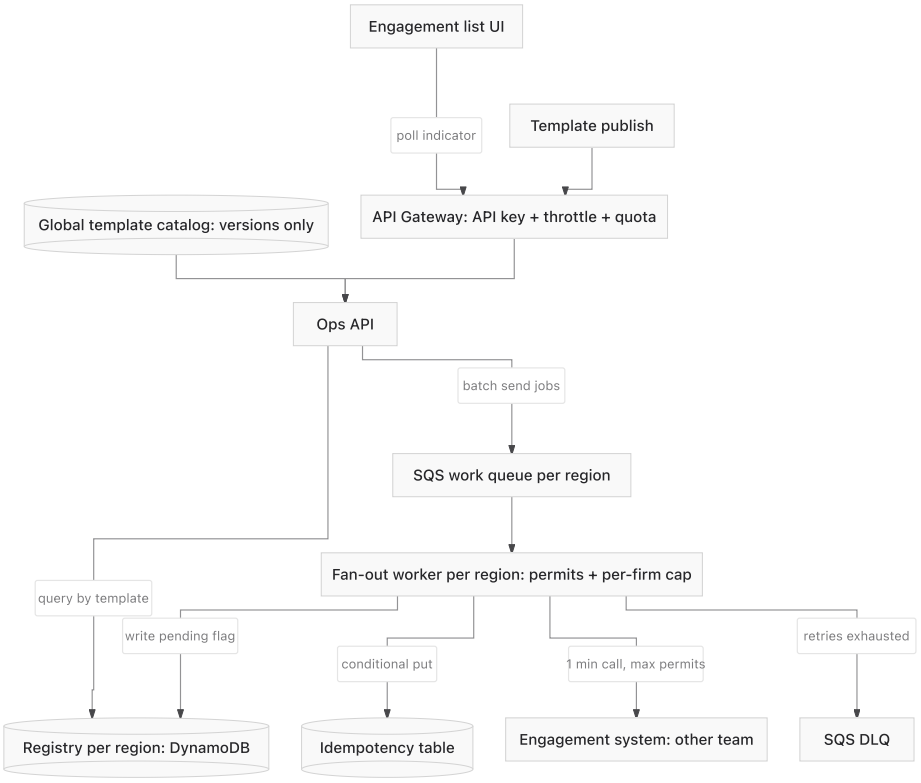
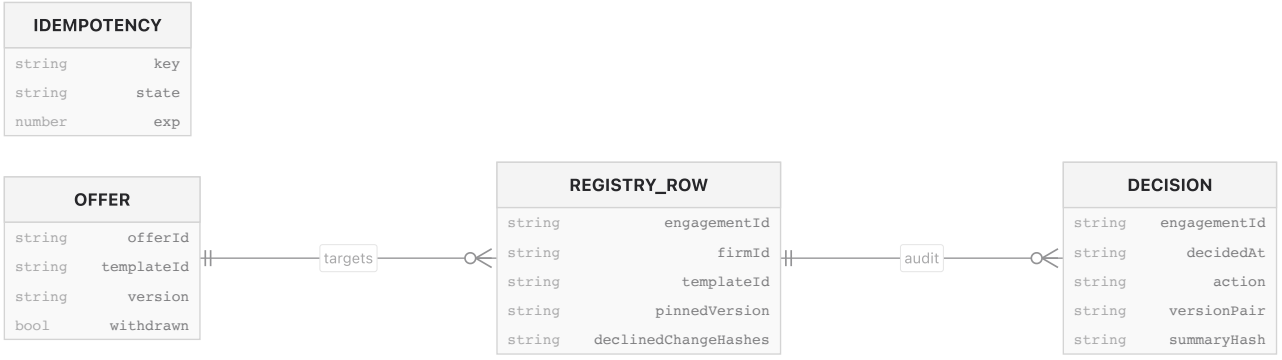
Template update fan-out: design

The one idea. The engagement system can only reveal an engagement's template version by loading it (about 1 minute, a hard constraint). Loading 800,000 engagements is 800,000 minutes (about 1.5 years serial; 2.8 days at 200 parallel loads). So **nothing in the publish path loads an engagement.** We keep a side-index (the *registry*) per region, fed by hooks, and a publish becomes an index query plus cheap writes. The slow call is used only for backfill and best-effort sync, always under a global permit cap.

1. Architecture and data ownership

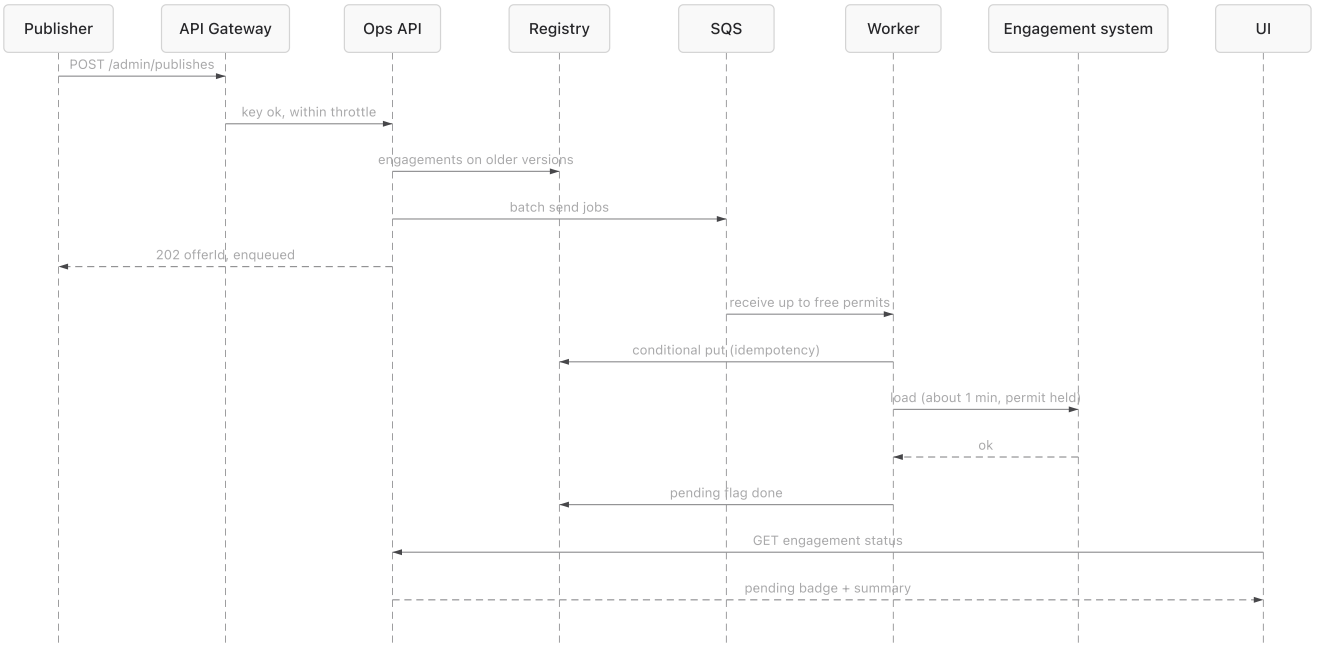


Data	Owner and place	Notes
Template versions, diffs, summaries	Existing template DB (global, shared)	Not firm-specific, so global is fine. Summaries are cached here per (product, from, to).
Engagements	Engagement system, customer databases (unchanged)	We never read them in the publish path.
Registry row: `engagementId, firmId, templateId, pinned version, declined change-units`	**Us, one DynamoDB per region**	Fed by create / apply / decline hooks. The UI reads this directly.
Offers (`templateId, version, withdrawn`) and decision audit	Us; offers global, decisions regional	A decision stores the version pair and summary hash the user saw.



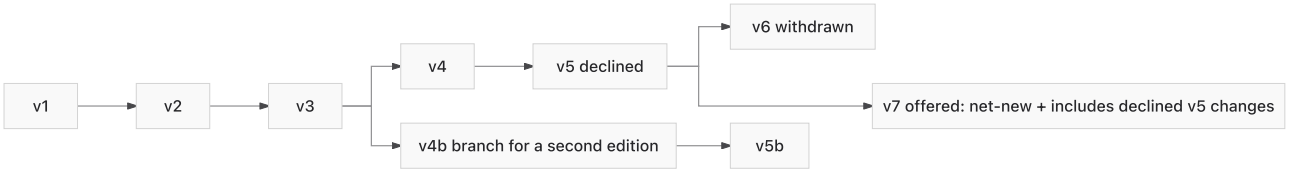
Data residency. EU and Canada firms must stay in-region; we assume everyone else lives in a default (US) region, so three stacks from one Terraform module. The global publish event carries only `(templateId, version, offerId)`. Each region fans out to its own registry, so engagement and firm ids never leave their region. The UI for an EU firm calls the EU stack.

Flow (diagram below): publish hook -> regional fan-out -> registry query by template -> queue jobs -> workers write the pending flag under a permit cap -> UI reads the flag.

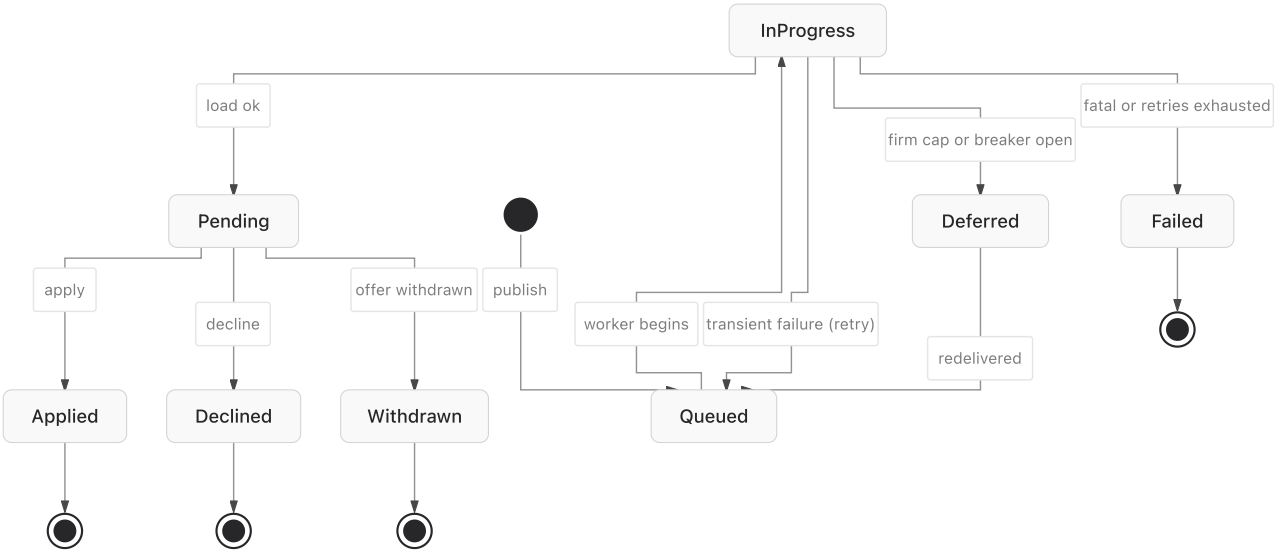


2. Correctness and production evolution

Versions are a DAG (markets branch, versions get withdrawn, v7 contains v5):



- **Pending target** = newest non-withdrawn version reachable from the engagement's version on its own market branch. When several updates accumulate, the user sees **one cumulative summary** (their version -> latest) plus the list of skipped intermediate versions, so a decision is on the real end state.
- **Withdrawal** is a tombstone on the offer; pending is recomputed from the next non-withdrawn descendant, and workers skip withdrawn offers.
- **What "declined" pins:** the set of **change-units** (hashes of individual changes), not a version number. Declining v5 then being offered v7 shows only net-new changes and says "includes previously declined v5 changes". Pinning a version would hide v5's changes inside v7, or re-ask for them as new.
- **Unreliable events:** hooks write through an outbox, delivery is at-least-once, every effect is idempotent on `(engagementId, templateId, targetVersion)` (a DynamoDB conditional put with a lease), and a **daily reconciler** compares registry rows against the engagement system's counts and repairs drift. Out-of-order events carry a version sequence; older ones are ignored.
- **Migration and backfill** (live, no window): hooks run dark -> backfill under the permit budget -> confirm-on-touch (an engagement is confirmed when someone opens it) -> per-tenant cutover at 99% or better coverage. Until a tenant is cut over the UI **fails quiet**: no badge rather than a false one. Backfill is bounded by the 1-minute load: see section 3.
- **Failure recovery:** retries with backoff, then a dead-letter queue we can replay; a crashed worker's idempotency lease expires and the key is retaken; the registry can be rebuilt from hooks plus a re-backfill.



3. Scale, cost and operations (arithmetic)

Given: 4,000 firms, 800,000 engagements, 40 products, weekly publishes (40 x 52/12 = **173 a month**), 1-minute load. OPTIMISTIC uses those. **MID and PESSIMISTIC are our assumptions**, as are the permit counts (the other team has not told us their capacity).

	OPTIMISTIC	MID	PESSIMISTIC
Firms / engagements	4,000 / 800,000	6,000 / 1,200,000	12,000 / 2,400,000
Downstream permits	200	100	50 (20 worst case)
Backfill = engagements x 60 s / permits	800,000 x 60 / 200 = 240,000 s = **2.8 days**	1,200,000 x 60 / 100 = **8.3 days**	2,400,000 x 60 / 50 = **33 days** (83 at 20)
Engagements affected per publish (/ 40)	20,000	30,000	60,000

Backfill always costs one load per existing engagement because the version is only readable by loading; listing IDs only saves finding them. For MID and PESSIMISTIC we backfill active firms first and rely on confirm-on-touch. **Fairness:** one firm can hold 40,000 engagements (median 40), so a per-firm cap of half the permits stops it starving the rest.

Cost. The brief leaves $X blank, so we model monthly caps of **$1,000 / $3,000 / $10,000**. Prices verified against aws.amazon.com list prices (2026-10-27, us-east-1): Fargate 1 vCPU + 2 GB about $36/task-month; DynamoDB on-demand $0.625/M writes and $0.125/M reads (the Nov 2024 50% cut); SQS $0.40/M requests at 3 per message; API Gateway $3.50/M; LLM summary $0.25 each; observability $60; 3 regions; UI = 5 users/firm x 440 views/user/month.

Monthly line	OPTIMISTIC	MID	PESSIMISTIC
Fargate, 2 tasks x 3 regions	216	216	216
DynamoDB writes (173 x affected; 3.46M / 5.19M / 10.4M)	2.2	3.2	6.5
DynamoDB reads (12.3M / 18.4M / 36.8M)	1.5	2.3	4.6
SQS (10.4M / 15.6M / 31.2M requests)	4.2	6.2	12.5
API Gateway (8.8M / 13.2M / 26.4M UI views)	31	46	92
LLM summaries (173 x $0.25)	43	43	43
Observability	60	60	60
TARGET total	**358**	**376**	**435**
LEAN total (1 task per region, no LLM)	207	225	285

TARGET is 36 / 13 / 4% of the $1k / $3k / $10k caps at OPTIMISTIC scale and 44 / 15 / 4% at PESSIMISTIC, so every combination fits with at least 2.2x headroom. Tiers: **LEAN** (rule-based summaries), **TARGET** (+ one cached LLM narrative per version pair), **CAP** (alarm at 80% of the cap and a kill-switch back to LEAN). The always-on workers are the biggest line; API Gateway grows with scale.

DynamoDB capacity mode: on-demand, decided on arithmetic. Provisioned capacity is ~3.4x cheaper per operation but only at 100% utilization ($0.18/M writes at $0.00065/WCU-h); it breaks even against on-demand at 29% sustained utilization. Our shape is a publish burst draining at up to 100-200 writes/s, then idle: provisioning for the peak costs ~200 WCU x $0.4745 x 3 regions ≈ $285/month vs $4-11 on-

demand for the same volumes — and risks throttling the backfill. Decision: **on-demand everywhere**, no free-tier dependency, no capacity planning; when steady, buy a **Database Savings Plan** (1-year, up to 18% off on-demand) rather than reserved provisioned capacity (which needs provisioned mode and >29% sustained utilization to ever pay off).

Caching, decided on cost. Added: an in-process 5 s TTL cache of the "offer withdrawn?" read (`CACHE_ENABLED`; $0; a withdrawal reaches a worker up to 5 s late, costing one harmless extra load). Rejected: an API Gateway cache (about $14.60 per region, $43.80 total, to save at most $2-7 of reads, and API Gateway still bills each request), DAX (about $88 per region, and it does not cache the consistent reads idempotency needs), a CDN (breaks residency). **Measured (one run each, LocalStack):** the cache cut offer reads by 93-98% but showed no reliable speed gain (drain was faster with it in OPTIMISTIC and MID and slower in PESSIMISTIC, and every cache-on run came second, so order is confounded). **Recommendation: ship with it off**; enable only if the read is a measured hot spot on real DynamoDB, after an alternating off/on/on/off test. See `results/locust/ANALYSIS.md`.

Observability and SLOs. Publish -> badge: 99% within 30 s (the brief says "within seconds"; the harness shows the control path, see section 5). Summary available: p95 within 15 min. Registry drift found by the reconciler under 0.1%. Alarms: queue age, DLQ depth above 0, permit utilization, per-firm starvation, 5xx at the gateway, cost at 80% of the cap. API Gateway also protects the ops API with an API key, a per-key throttle (50 req/s) and a daily quota.

4. Human-readable summaries: generation and evaluation

Generate once per (product, from-version, to-version) and cache globally, since summaries are not firm-specific; inference cost is then noise (173 pairs a month).

1. The existing JSON diff is turned into **typed change-units** (added, removed, changed, with section path), each hashed.
2. A **deterministic renderer** writes the user-facing summary from those units (grouped by section, plain language). Numbers and names come only from the diff.
3. An **optional LLM narrative** may polish wording. It is display-only, labelled as such, and never the source of any figure.

Defending a March summary in November. Each summary is stored immutably (versioned object storage with object lock) with provenance: both template version ids and content hashes, diff hash, renderer version, model id and version, prompt version, the exact output, and a timestamp. A decision record stores the version pair and summary hash the user saw. Re-rendering the deterministic part from the stored diff must reproduce it byte for byte; the narrative is reproduced by showing the stored output, not by regenerating it.

Evaluation. (a) Mechanical check: every number, name and section in a narrative must appear in the diff, otherwise the narrative is dropped and the deterministic summary is shown. (b) A golden set of real template pairs, rerun on every renderer, prompt or model change. (c) Human review of a sample, weighted to regulated content, with reviewers' edits fed back to the golden set. (d) Pinned model versions; a change is a release, not a config tweak.

5. Tradeoffs, assumptions, challenge, risk, what we left out

- **Requirement we would challenge: "real time, within seconds" for everything.** The badge can be seconds (a registry read); the summary can be minutes. Chasing seconds for the summary, or for engagements not yet in the registry, forces loads we cannot afford. We would also ask for the downstream's real capacity before committing to dates.
- **Riskiest to get wrong:** the index drifting from the truth, plus the backfill (it depends on a 1-minute call we do not own), then the decline semantics. Mitigation: reconciler, confirm-on-touch, fail-quiet, 99% coverage gate.
- **Assumptions:** hooks exist on both systems (the brief allows it); the engagement-list UI can call our indicator service instead of the engagement system; a load also exposes applied/declined history, otherwise history before the hooks is "unknown" and fails quiet; the engagement stores the creation version, so hooks are the source of truth going forward.
- **Left out on purpose:** applying template content (out of scope); a real engagement-system integration (stubbed, 1 minute compressed); real LLM calls (the design is specified, not built); multi-region failover; authorization beyond the API key; production-grade UI. The UI here is a test harness.
- **Part 2 evidence** (all under `results/`): 30 unit and integration tests pass (8 on LocalStack); Bruno 12 requests, 18/18 assertions; Playwright 3/3 with screenshots and recorded API calls; Terraform applied on LocalStack (queues, tables, alarms); six load runs (3 scenarios x cache off/on) in `results/locust/REPORT.md`, all with peak concurrency within permits, zero duplicate downstream calls and an empty DLQ. Measured drain was 2.5x to 6.3x the Little's Law prediction (it is a lower bound), and the 100 ms status-read target was met in 2 of 6 runs against LocalStack. Limits: the load run models the 1-minute path, not DynamoDB write speed; the harness never reached the permit cap at 100-200 permits, so real-time extrapolations assume the permits are the bottleneck; LocalStack does not enforce API keys or throttling; DynamoDB capacity-mode decision (on-demand) is arithmetic against verified list prices, the other prices are list and untested in billing.
- **Best price and quality, by the results:** 2 Fargate tasks per region plus API Gateway (TARGET, $358-435 a month, 4-44% of the assumed caps); no DAX, no gateway cache; in-process cache off by default; DynamoDB on on-demand capacity; first fixes before production are a one-read status path and a measurement against real DynamoDB.